Overview of Long short-term memory

Subject: Long short-term memory

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$h_t = o_t \odot \tanh(c_t)$$

1.

W ∈ R h × d {\displaystyle W\in \mathbb {R} ^{h\times d}} , U ∈ R h × h {\displaystyle U\in \mathbb {R} ^{h\times h}} and b ∈ R h {\displaystyle b\in \mathbb {R} ^{h}} : weight matrices and bias vector parameters which need to be learned during training

2.

Development of variants In 2005 Graves and Schmidhuber published LSTM with full backpropagation through time and bidirectional LSTM. In 2006 Graves, Fernandez, Gomez, and Schmidhuber introduced a new error function for LSTM: Connectionist Temporal Classification (CTC) for simultaneous alignment and recognition of sequences. In 2014 Kyunghyun Cho an others published a simplified variant of the forget gate LSTM called Gated recurrent unit (GRU). In 2015 Srivastava, Greff, and Schmidhuber used LSTM principles to create the Highway network, a feedforward neural network with hundreds of layers, much deeper than previous networks. Concurrently, the ResNet architecture was developed. It is equivalent to an open-gated or gateless highway network. A modern upgrade of LSTM called xLSTM is published by a team led by Sepp Hochreiter. One of the 2 blocks (mLSTM) of the architecture are parallelizable like the Transformer architecture, the other ones (sLSTM) allow state tracking.

3.

where the initial values are c 0 = 0 {\displaystyle c_{0}=0} and h 0 = 0 {\displaystyle h_{0}=0} and the operator ■ {\displaystyle \odot } denotes the Hadamard product (element-wise product). The subscript t {\displaystyle t} indexes the time step.

4.

CTC score function Many applications use stacks of LSTM RNNs and train them by connectionist temporal classification (CTC) to find an RNN weight matrix that maximizes the probability of the label sequences in a training set, given the corresponding input sequences. CTC achieves both alignment and recognition.

5.

Variants In the equations below, the lowercase variables represent vectors. Matrices W q {\displaystyle W_{q}} and U q {\displaystyle U_{q}} contain, respectively, the weights of the input and recurrent connections, where the subscript q {\displaystyle _{q}} can either be the input gate i {\displaystyle i} , output gate o {\displaystyle o} , the forget gate f {\displaystyle f} or the memory cell c {\displaystyle c} , depending on the activation being calculated. In this section, we are thus using a "vector notation". So, for example, c t ∈ R h {\displaystyle c_{t}\in \mathbb {R} ^{h}} is not just one unit of one LSTM cell, but contains h {\displaystyle h} LSTM cell's units.

6.

Long short-term memory (LSTM) is a type of recurrent neural network (RNN) aimed at mitigating the vanishing gradient problem commonly encountered by traditional RNNs. Its relative insensitivity to gap length is its advantage over other RNNs, hidden Markov models, and other sequence learning methods. It aims to provide a short-term memory for RNN that can last thousands of timesteps (thus "long short-term memory"). The name is made in analogy with long-term memory and short-term memory and their relationship, studied by cognitive psychologists since the early 20th century. An LSTM unit is typically composed of a cell and three gates: an input gate, an output gate, and a forget gate. The cell remembers values over arbitrary time intervals, and the gates regulate the flow of information into and out of the cell. Forget gates decide what information to discard from the previous state, by mapping the previous state and the current input to a value between 0 and 1. A (rounded) value of 1 signifies retention of the information, and a value of 0 represents discarding. Input gates decide which pieces of new information to store in the current cell state, using the same system as forget gates. Output gates control which pieces of information in the current cell state to output, by assigning a value from 0 to 1 to the information, considering the previous and current states. Selectively outputting relevant information from the current state allows the LSTM network to maintain useful, long-term dependencies to make predictions, both in current and future time-steps. LSTM has wide applications in classification, data processing, time series analysis tasks, speech recognition, machine translation, speech activity detection, robot control, video games, healthcare, energy forecasting.

7.

f t = σ g (W f ∗ x t + U f ∗ h t − 1 + V f ■ c t − 1 + b f) i t = σ g (W i ∗ x t + U i ∗ h t − 1 + V i ■ c t − 1 + b i) c t = f t ■ c t − 1 + i t ■ σ c (W c ∗ x t + U c ∗ h t − 1 + b c) o t = σ g (W o ∗ x t + U o ∗ h t − 1 + V o ■ c t + b o) h t = o t ■ σ h (c t) {\displaystyle {\begin{aligned}f_{t}&=\sigma _{g}(W_{f}*x_{t}+U_{f}*h_{t-1}+V_{f}\odot c_{t-1}+b_{f})\\i_{t}&=\sigma _{g}(W_{i}*x_{t}+U_{i}*h_{t-1}+V_{i}\odot c_{t-1}+b_{i})\\c_{t}&=f_{t}\odot c_{t-1}+i_{t}\odot \sigma _{c}(W_{c}*x_{t}+U_{c}*h_{t-1}+b_{c})\\o_{t}&=\sigma _{g}(W_{o}*x_{t}+U_{o}*h_{t-1}+V_{o}\odot c_{t}+b_{o})\\h_{t}&=o_{t}\odot \sigma _{h}(c_{t})\end{aligned}}}

8.

Further reading Monner, Derek D.; Reggia, James A. (2010). "A generalized LSTM-like training algorithm for second-order recurrent neural networks" (PDF). Neural Networks. 25 (1): 70–83. doi:10.1016/j.neunet.2011.07.003. PMC 3217173. PMID 21803542. High-performing extension of LSTM that has been simplified to a single node type and can train arbitrary architectures Gers, Felix A.; Schraudolph, Nicol N.; Schmidhuber, Jürgen (Aug 2002). "Learning precise timing with LSTM recurrent networks" (PDF). Journal of Machine Learning Research. 3: 115–143. Gers, Felix (2001). "Long Short-Term Memory in Recurrent Neural Networks" (PDF). PhD thesis. Abidogun, Olusola Adeniyi (2005). Data Mining, Fraud Detection and Mobile Telecommunications: Call Pattern Analysis with Unsupervised Neural Networks. Master's Thesis (Thesis). University of the Western Cape. hdl:11394/249. Archived (PDF) from the original on May 22, 2012. original with two chapters devoted to explaining recurrent neural networks, especially LSTM.

9.

Training An RNN using LSTM units can be trained in a supervised fashion on a set of training sequences, using an optimization algorithm like gradient descent combined with backpropagation through time to compute the gradients needed during the optimization process, in order to change each weight of the LSTM network in proportion to the derivative of the error (at the output layer of the LSTM network) with respect to corresponding weight. A problem with using gradient descent for standard RNNs is that error gradients vanish exponentially quickly with the size of the time lag between important events. This is due to lim n → ∞ W n = 0 {\displaystyle \lim _{n\to \infty }W^{n}=0} if the spectral radius of W {\displaystyle W} is smaller than 1. However, with LSTM units, when error values are back-propagated from the output layer, the error remains in the LSTM unit's cell. This "error carousel" continuously feeds error back to each of the LSTM unit's gates, until they learn to cut off the value.